

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы.

Студенты г. 4341

Руководитель

К.А. Берсенов

А.С. Большакова

Д.Р. Валитов

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студенты К.А. Берсенев, А.С. Большакова, Д.Р. Валитов

Группа: 4341

Тема практики: Генетические алгоритмы

Задание на практику:

Задача о назначениях

Пусть имеется N работ и N кандидатов на выполнение этих работ, причем назначение j -й работы i -му кандидату требует затрат $c_{ij} > 0$. Необходимо назначить каждому кандидату по работе, чтобы минимизировать суммарные затраты. При этом каждый кандидат может быть назначен на одну работу, а каждая работа может выполняться только одним кандидатом.

Сроки прохождения практики: 06.07.2026-18.07.2026

Дата сдачи отчета: 16.07.2026

Дата защиты отчета: 17.07.2026

Студент

К.А. Берсенев
А.С. Большакова
Д.Р. Валитов

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

Разработана программа для решения задачи о назначениях: имеются N работ и N кандидатов, назначение j -й работы i -му кандидату требует затрат $c_{ij} > 0$. Необходимо минимизировать суммарные затраты при условии, что каждый кандидат выполняет одну работу, а каждая работа — только одним кандидатом.

В основе решения — генетический алгоритм с хромосомой-перестановкой. Реализованы формирование популяции, анализ решений целевой функцией, скрещивание и мутация с заданной вероятностью. Программа написана на Python и снабжена графическим интерфейсом, позволяющим задавать размерность, генерировать данные, загружать их из файла и управлять параметрами ГА (размер популяции, число поколений, вероятности мутации и скрещивания).

Предусмотрена визуализация хода поиска: для каждой популяции выводится текущее и глобальное лучшее решение и значение целевой функции, строится график целевой функции, доступен пошаговый режим с возможностью возврата на несколько шагов назад и переход к финальному результату.

SUMMARY

A program has been developed to solve the assignment problem: there are N jobs and N candidates, and assigning the j -th job to the i -th candidate incurs a cost $c_{ij} > 0$. The goal is to minimize the total cost under the constraint that each candidate is assigned to exactly one job and each job is performed by exactly one candidate.

The solution is based on a genetic algorithm with permutation-based chromosome encoding. The implementation includes population initialization, solution evaluation and selection for subsequent operations, crossover and mutation with specified probabilities. The program is written in Python and features a graphical user interface that allows the user to set the problem size, generate random data, load data from a file, and adjust GA parameters (population size, number of generations, crossover and mutation probabilities).

Visualization of the search process is provided: for each population, the current best solution and the global best solution along with their objective function values are displayed, a convergence graph of the fitness function is plotted incrementally, and a step-by-step mode is available with the ability to go back several steps as well as to jump directly to the final result.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1 ПОСТАНОВКА ЗАДАЧИ	7
1.1 Предметная область	7
1.2 Задачи практики	7
1.3 Этапы работы.....	8
1.4 Требования к реализации	8
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	10
2.1. Организация работы и распределение подзадач.....	10
2.2. Структура данных и хранение состояния.....	10
2.3. Генетический алгоритм. Операторы и параметры.	11
2.4. Синхронизация GUI и ГА.....	11
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	13
4 ОТЗЫВ РУКОВОДИТЕЛЯ	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16
ПРИЛОЖЕНИЕ А	17

ВВЕДЕНИЕ

Целью практики является разработка программы для решения задачи о назначениях на основе генетического алгоритма, обеспечивающего визуализацию процесса поиска и интерактивное управление ходом вычислений. Для достижения поставленной цели необходимо реализовать три функциональных модуля — графический интерфейс пользователя, ядро генетического алгоритма и контроллер, отвечающий за синхронизацию между ними, — которые будут разрабатываться параллельно тремя участниками бригады. В ходе выполнения практики планируется изучить принципы построения генетических алгоритмов, методы отбора, скрещивания и мутации для задач с перестановками, а также освоить инструменты создания оконных приложений на Python с библиотекой Tkinter и встраивания графиков Matplotlib. Итоговый продукт должен позволять задавать входные данные, настраивать параметры алгоритма, пошагово наблюдать за эволюцией популяции, отслеживать изменение целевой функции и возвращаться к предыдущим состояниям. Более детальное описание каждого модуля, их взаимодействия и реализованных операторов представлено в соответствующих разделах отчёта.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются в основном в задачах дискретной и комбинаторной оптимизации, где классические точные методы становятся слишком трудоёмкими из-за резкого роста числа возможных вариантов при увеличении размерности задачи. Также применяются в ситуациях, где нужно одновременно учитывать несколько целей (например, минимизировать затраты и максимизировать качество), и когда целевая функция не имеет простой математической формулы или содержит много локальных максимумов и минимумов. Задача о назначении кандидатов на работы относится именно к комбинаторному типу и является типичной для таких прикладных областей, как кадровое планирование, распределение производственных ресурсов и оперативное управление, где необходимо быстро находить приемлемое решение без потери качества.

1.2 Задачи практики:

1. Изучить теоретические основы задачи о назначениях, формальную постановку и существующие методы решения.
2. Изучить принципы работы генетических алгоритмов: формирование популяции решений, операторы отбора, скрещивания, мутации.
3. Разбить задание на подзадачи и этапы, распределить роли в команде.
4. Разработать и реализовать на языке Python ядро генетического алгоритма для задачи о назначениях (инициализация популяции, анализ целевой функцией, скрещивание решений, мутация, анализ).
5. Разработать графический интерфейс пользователя (GUI) на базе Tkinter, обеспечивающий ввод данных, настройку параметров алгоритма и визуализацию процесса поиска.
6. Обеспечить пошаговый режим работы алгоритма с возможностью возврата на несколько шагов назад и перехода к финальному решению.

7. Провести эксперименты с различными настройками ГА (размер популяции, число поколений, вероятности скрещивания и мутации) и проанализировать их влияние на результат.

В процессе работы были изучены следующие технологии и инструменты:

- язык программирования Python (библиотеки Tkinter для GUI, Matplotlib для построения графиков, NumPy для работы с матрицами);
- принципы проектирования пользовательских интерфейсов и встраивания графиков в приложение;
- методы работы с файловым вводом-выводом для загрузки данных;
- подходы к построению и отладке генетических алгоритмов.

1.3 Этапы работы

1. Формирование бригады, выбор задания и ЯП, распределение ролей в бригаде
2. Демонстрация прототипа GUI и плана по решению задачи (описание формата данных, используемой функции качества, и.т.д.)
3. Промежуточная сдача программы (несколько итераций). На данном этапе должен быть частично-функционирующий GUI, работающий алгоритм, но возможно, требующий корректировок.
4. Демонстрация финальной версии программы и отчета.

1.4 Требования к реализации

- Программа должна иметь GUI.
- Должна быть возможность задать данные через GUI, чтение из файла, случайную генерацию.
- Алгоритмы реализуются самостоятельно без использования готовых решений.
- Настройкой параметров алгоритмов должна производиться пользователем.

- Пошаговая визуализация поиска решения (как меняется аппроксимирующая функция, текущий экстремум, текущее решение оптимизационной задачи в зависимости от популяции). Должен быть выбор переключения между разными текущими решениями.
- Возможность перейти к конечному решению пропустив отображение всех шагов.
- Должен присутствовать график изменения функции качества решения от номера популяции. График дополняется с каждым шагом алгоритма.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Организация работы и распределение подзадач

В работе участвовала бригада из трёх человек. Для эффективной параллельной разработки программа была разделена на три функциональных модуля, каждый из которых закреплён за отдельным участником. Первому участнику была выдана задача разработки графического интерфейса на основе Tkinter, включая создание окон ввода параметров, отображение матрицы затрат в виде таблицы, визуализацию текущего решения и графика сходимости, а также панель навигации с кнопками управления. Вторым участником отвечает за реализацию ядра генетического алгоритма: функции создания начальной популяции, расчёта целевой функции и приспособленности, отбора методом масштабирования, упорядоченного скрещивания, мутации обменом. Третьему участнику была поручена разработка контроллера — связующего слоя между GUI и ядром алгоритма, который управляет состоянием приложения, хранит историю популяций для возврата на несколько шагов назад, обрабатывает команды пошаговой навигации и обновляет данные на экране. Такое разделение позволило вести разработку независимо, используя общий контракт в виде структур данных и названий функций, а сборка итогового приложения производится путём импорта всех трёх модулей в единую точку входа.

2.2. Структура данных и хранение состояния

В основе программы лежат следующие структуры данных:

Матрица затрат — двумерный список размером $N \times N$, содержащий положительные числа $c[i][j]$ (затраты на назначение j -й работы i -му кандидату).

Хромосома (решение) — список целых чисел длины N , представляющий перестановку. Индекс в списке соответствует кандидату, значение — номеру назначенной работы.

Популяция — список хромосом (размер популяции задаётся пользователем).

История популяций — для реализации возврата на несколько шагов назад хранятся последние три поколения (как список из трёх популяций). При каждом новом шаге самая старая популяция удаляется, новая добавляется.

Глобальное лучшее решение — хромосома с минимальным значением целевой функции за всё время работы алгоритма, сохраняется и обновляется на каждом поколении.

2.3. Генетический алгоритм. Операторы и параметры.

Целевая функция — сумма затрат для заданной хромосомы:

$$\text{Fitness} = \sum_{i=0}^{n-1} \text{matrix}[i][\text{chromosome}[i]]$$

Приспособленность — величина, обратная целевой функции (для преобразования задачи минимизации в задачу максимизации при отборе):

$$\text{FitnessScore} = \frac{1}{\text{Fitness} + 1}$$

Метод отбора — метод масштабирования приспособленности: вероятность выбора особи пропорциональна её значению приспособленности.

Скрещивание — упорядоченное. Сохраняет относительный порядок генов родителей, гарантирует отсутствие повторений.

Мутация — обмен двух случайных позиций в хромосоме.

Элитизм — при формировании нового поколения 2% лучших особей из предыдущей популяции переносятся без изменений.

Оставшиеся особи создаются следующим образом:

- выбор двух родителей методом рулетки;
- скрещивание (при отсутствии скрещивания потомки идентичны родителям);
- мутация полученного потомка.
- Остальные параметры (размер популяции, число поколений, вероятности) настраиваются пользователем через GUI.

2.4. Синхронизация GUI и ГА

Контроллер выступает в качестве связи между пользовательским интерфейсом и ядром алгоритма. Его функции:

Инициализация — получение от GUI параметров (N , размер популяции, число поколений, вероятности, матрица затрат), запуск первого поколения через ядро ГА.

Хранение состояния — поддержка текущего индекса поколения, истории популяций, глобального лучшего решения.

Обработка команд навигации:

- «Шаг вперёд» — запуск одной итерации ГА (отбор, скрещивание, мутация, формирование нового поколения), обновление истории, передача обновлённых данных в GUI.
- «Назад» — откат к предыдущему поколению (из истории), перерисовка интерфейса с сохранёнными данными.
- «В конец» — выполнение всех оставшихся поколений без промежуточных остановок, отображение финального результата.

Обновление GUI — после каждого изменения состояния контроллер передаёт в интерфейс:

- текущую популяцию (для отображения лучшего решения в ней);
- глобальное лучшее решение и его стоимость;
- новые точки для графика сходимости;
- статистику (номер поколения, число скрещиваний, число мутаций).

Синхронизация организована так, что ядро ГА не содержит кода визуализации и работает независимо от GUI. Контроллер вызывает функции ядра, получает результаты и передаёт их в интерфейс через заранее определённые методы обновления.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В данной главе опишите технологии, которые использовались вами при выполнении задач в рамках практики.

Это могут быть языки программирования, списки конкретных библиотек / ПО и целей / мест их применения.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно
добавить формулировку:

По результатам работы учебная практика оценена на <ваша оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв

о прохождении учебной практики

«ФИО студента», студент(ка) группы «номер группы», второго
курса бакалавриата Санкт-Петербургского электротехнического
университета «ЛЭТИ» им В.И. Ульянова, проходил(а) учебную практику
по теме «тема практики» с дата начала по дата окончания.

В течение практики обучающийся(аяся) <ФИО студента> выполнял(а) задание на практику>.

В результате практики обучающийся(аяся) <результат практики>. Получаемую работу обучающийся(аяся) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата>

<подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

**СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (ЗАГОЛОВОК
ВТОРОГО УРОВНЯ)**

1. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 28.04.2021).
2. Бобкова, О.В., Давыдов, С.А., Ковалева, И.А., Плагиат как гражданское правонарушение / О.В. Бобкова, С.А. Давыдов, И.А. Ковалева // Патенты и лицензии. – 2016. – № 7. – С. 31-37.

ПРИЛОЖЕНИЕ А (ЗАГОЛОВОК ВТОРОГО УРОВНЯ)